Model-Based Volatility Estimation

In this example we obtain volatility estimates by creating parametric time-series models. One disadvantage of the previous approach for volatility estimation using the exponentially-weighted moving average is that it does not provide volatility forecasts. However, a model-based approach can be used to create volatility forecasts.

This example focuses on the `ARIMA` and `GARCH` time-series models.

Table of Contents

Load the commodity data.

```
load( "Commodities.mat" )
commodityNames = string( InferredVolatility.Properties.VariableNames );
```

Compute the principal components of the inferred volatility.

First, normalize the volatility series and store the mean and standard deviation for later use.

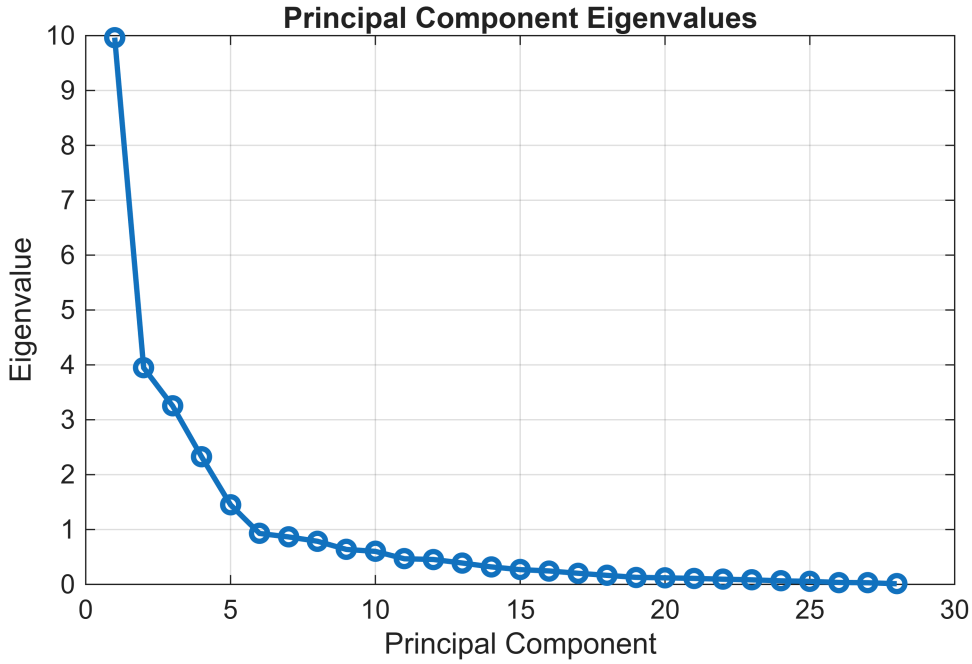
```
[normalizedVolatility, mu, sigma] = normalize( InferredVolatility.Variables,
"zscore" );
```

Next, compute the principal components using the `pca` function.

```
[coeffs, scores, eigVals] = pca( normalizedVolatility );
```

Visualize the eigenvalues of the covariance matrix of the normalized volatilities.

```
figure
plot( eigVals, "o-", "LineWidth", 2 )
xlabel( "Principal Component" )
ylabel( "Eigenvalue" )
title( "Principal Component Eigenvalues" )
grid( "on" )
```



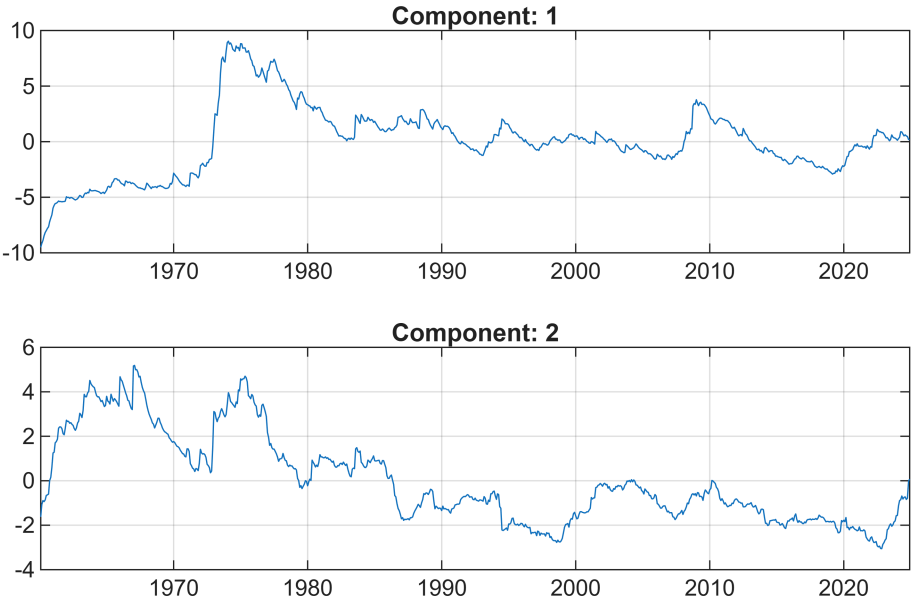
We see that most of the variance in the data has been captured by the first few principal components.

Visualize the first few principal components.

```
numComponents = 2;

figure
tiledlayout( "flow" )

for k = 1:numComponents
    nexttile
    plot( InferredVolatility.Date, scores(:, k ) )
    title( "Component: " + k )
    grid( "on" )
end
```



Model the principal components.

Rather than creating many separate time-series models for each commodity, an efficient approach is to build models only for the first few principal components. Forecasts from these models can then be back-transformed into the original domain.

Create and fit a time-series model for the first principal component.

```
firstComponent = scores(:, 1);

firstComponentModel = arima( "ARLags", 1, ...
    "MALags", 1, ...
    "Distribution", "t", ...
    "Variance", garch( 1, 1 ) );
firstComponentModel = estimate( firstComponentModel, firstComponent );
```

```
ARIMA(1,0,1) Model (t Distribution):

             Value      StandardError    TStatistic      PValue
            _________    _____________    __________    _________

Constant    -0.062171      0.0062277        -9.983      1.8089e-23
AR{1}         0.98299      0.0019881        494.44               0
MA{1}         0.12742       0.025704         4.957      7.1586e-07
DoF            2.4855        0.21456        11.584      4.9579e-31


GARCH(1,1) Conditional Variance Model (t Distribution):

             Value      StandardError    TStatistic      PValue
            _________    _____________    __________    _________
```

Constant	0.0033784	0.0017628	1.9165	0.055304
GARCH{1}	0.91591	0.028769	31.836	2.025e-222
ARCH{1}	0.046599	0.02028	2.2978	0.021572
DoF	2.4855	0.21456	11.584	4.9579e-31

Create and fit a time-series model for the second principal component.

```
secondComponent = scores(:, 2);
secondComponentModel = arima("ARLags", 1, ...
    "MALags", 1, ...
    "Distribution", "t", ...
    "Variance", garch( "ARCHLags", 1, ...
    "Constant", 0.01, ...
    "GARCHLags", 1 ) );
secondComponentModel = estimate( secondComponentModel, secondComponent );
```

```
ARIMA(1,0,1) Model (t Distribution):
```

	Value	StandardError	TStatistic	PValue
Constant	-0.023706	0.0043982	-5.39	7.0461e-08
AR{1}	0.98196	0.0024931	393.87	0
MA{1}	0.084913	0.025156	3.3755	0.00073684
DoF	2.1644	0.087826	24.644	4.2345e-134

```
GARCH(1,1) Conditional Variance Model (t Distribution):
```

	Value	StandardError	TStatistic	PValue
Constant	0.01	0	Inf	0
GARCH{1}	0.86587	0.058954	14.687	7.7774e-49
ARCH{1}	0.089066	0.044494	2.0018	0.04531
DoF	2.1644	0.087826	24.644	4.2345e-134

Create simulated component values from the fitted models.

```
numForecastSteps = 60;
simFirstComponent = simulate( firstComponentModel, numForecastSteps, "Y0",
firstComponent );
simSecondComponent = simulate( secondComponentModel, numForecastSteps, "Y0",
secondComponent );
simComponents = [simFirstComponent, simSecondComponent];
```

Back-transform the simulated component values.

Transform the simulation results in the principal component domain back to the original volatility domain, adjusting for the normalization step performed above. Since we have only modelled the first two principal components, this transformation is an approximation to the volatility. Modelling more of the principal

components would lead to a more accurate volatility approximation at the cost of more modelling, simulation and execution time.

```
approxVolatility = mu + sigma .* (simComponents * coeffs(:, 1:2).');
approxVolatility = [InferredVolatility{end, :}; approxVolatility];
```

Convert the volatility approximations to the timetable format.

```
simDates = InferredVolatility.Date(end) + calmonths(0:numForecastSteps).';
approxVolatility = array2timetable( approxVolatility, ...
    "RowTimes", simDates, "VariableNames", commodityNames );
approxVolatility.Properties.DimensionNames(1) = "Date";
```

Visualize the volatility simulation results.

```
selectedCommodity = commodityNames(7);

figure
plot( InferredVolatility.Date, InferredVolatility.(selectedCommodity) )
hold on
plot( approxVolatility.Date, approxVolatility.(selectedCommodity) )
xlabel( "Date" )
ylabel( "Volatility" )
title( "Model-Based Volatility Simulation" )
subtitle( "Selected Commodity: " + selectedCommodity )
grid on
```